

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA0603	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Shaik Basheer	Reg. No.:	192465049
Section / Batch:	CSE(CS)	Date:	19-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Maximum Product Subarray

Given an array of integers, find the contiguous subarray that has the maximum product. The array may

contain negative numbers, which can affect the product value. The goal is to determine the maximum possible product. Use Dynamic Programming to compute the result efficiently.

Sample Input:

n = 5

arr = 2 3 -2 4 -1

Expected Output:

Max Product = 48

Assessment – Challenging Task: Maximum Product Subarray

1. Problem Statement

Given an array of integers, find the **contiguous subarray with the maximum product**. The array may contain positive numbers, negative numbers, and zeros. Negative numbers can change the maximum product because multiplying two negative numbers produces a positive value.

For example:

Input:

$n = 5$

$arr = 2\ 3\ -2\ 4\ -1$

Output:

Max Product = 48

The subarray producing the maximum product is:

$2 \times 3 \times -2 \times 4 \times -1 = 48$

2. Challenging Task

Write an efficient program using **Dynamic Programming** to calculate the maximum product subarray.

The solution must correctly handle:

- Positive numbers
- Negative numbers
- Zero
- Multiple negative numbers
- Arrays containing only negative numbers

3. Algorithm

1. Initialize max_product and min_product with the first element.
2. Store the overall maximum product in result.
3. Traverse the array from the second element.
4. If the current number is negative, swap max_product and min_product.
5. Update max_product using the larger value between:
 - Current element
 - Current element $\times$ previous maximum product
6. Update min_product using the smaller value between:
 - Current element
 - Current element $\times$ previous minimum product
7. Update result with the maximum value.

8. Return result.

4. C Program

```
main.c
1  #include <stdio.h>
2  int max(int a, int b)
3  {
4      return (a > b) ? a : b;
5  }
6  int min(int a, int b)
7  {
8      return (a < b) ? a : b;
9  }
10 int maxProductSubarray(int arr[], int n)
11 {
12     int max_product = arr[0];
13     int min_product = arr[0];
14     int result = arr[0];
15     for (int i = 1; i < n; i++)
16     {
17         if (arr[i] < 0)
18         {
19             int temp = max_product;
20             max_product = min_product;
21             min_product = temp;
22         }
23         max_product = max(arr[i], max_product * arr[i]);
24         min_product = min(arr[i], min_product * arr[i]);
25         result = max(result, max_product);
26     }
27     return result;
28 }
29 int main()
30 {
31     int n;
32     printf("Enter number of elements: ");
33     scanf("%d", &n);
34     int arr[n];
35     printf("Enter array elements: ");
36     for (int i = 0; i < n; i++)
37     {
38         scanf("%d", &arr[i]);
39     }
40     int result = maxProductSubarray(arr, n);
41
42     printf("Max Product = %d\n", result);
43
44     return 0;
45 }
```

Output:

```
Enter number of elements: 5
Enter array elements: 2 3 -2 4 -1
Max Product = 48

...Program finished with exit code 0
Press ENTER to exit console.
```

7. Why min_product is Required

This is the challenging part of the problem.

For example:

$$-2 \times -3 = 6$$

A negative minimum product can become the **maximum product** when multiplied by another negative number.

Therefore, we maintain both:

max_product → maximum product ending at current position

min_product → minimum product ending at current position

This allows the algorithm to correctly handle negative numbers.

8. Complexity Analysis

- **Time Complexity: $O(n)$**

The array is traversed only once.

- **Space Complexity: $O(1)$**

Only a few variables are used apart from the input array.

9. Assessment Result

The Maximum Product Subarray problem was solved using Dynamic Programming. The algorithm maintains both maximum and minimum products because a negative value can convert the minimum product into the maximum product. The optimized solution runs in **$O(n)$ time and $O(1)$ extra space**, making it efficient for large arrays.